

# SNAKE GAME

## Java Swing Project

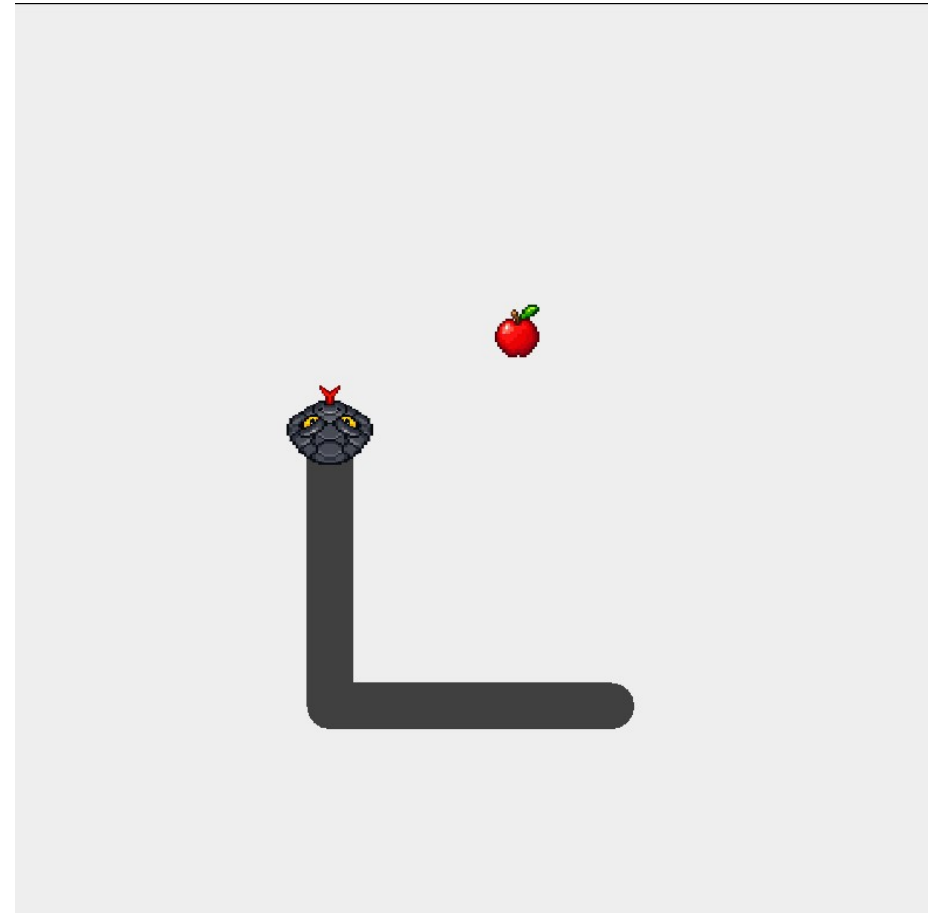
A classic Snake game built with Swing, object-oriented design, keyboard interaction, collision detection, and a Thread-based game loop.

JAVA

SWING

THREAD

OOP



*Real screenshot from the current game version*

# Game Overview

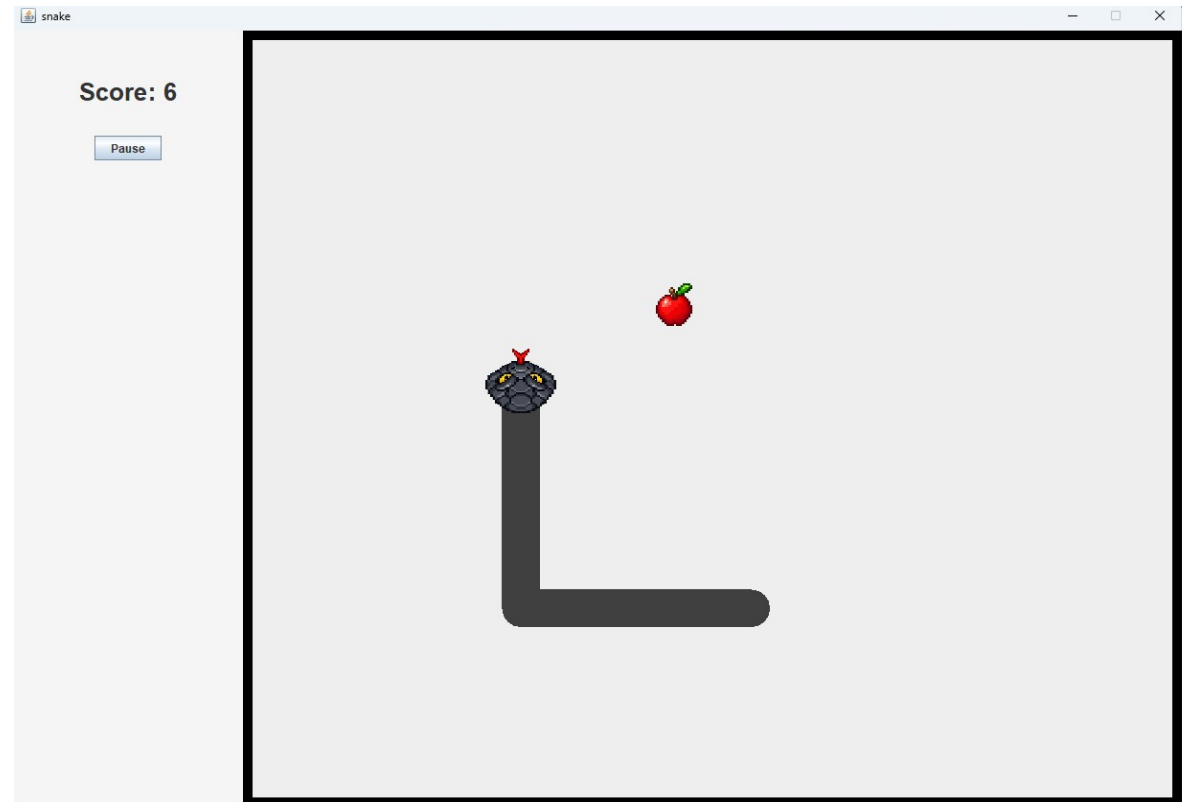
What the player sees and what the game is trying to achieve

## CORE GAMEPLAY

**Eat apples. Grow longer.  
Stay alive.**

- Move the snake across a fixed grid using W, A, S and D.
- Eating an apple increases the score and grows the body by one segment.
- The apple respawns at a random free position.
- The game ends when the snake hits a wall or its own body.

## Typical Game State



# Controls & Player Experience

The game starts from an instruction overlay and can be paused at any time

**W**

**Move Up**

Keyboard control

**A**

**Move Left**

Keyboard control

**S**

**Move Down**

Keyboard control

**D**

**Move Right**

Keyboard control

**SPACE**

**Pause / Resume**

Keyboard + menu button

**1**

## Instruction Screen

The overlay explains movement, pause/resume, and the goal. The game does not start until the player presses "Start Game".

**2**

## Active Game

The score and pause button remain visible in the side menu while the snake moves automatically.

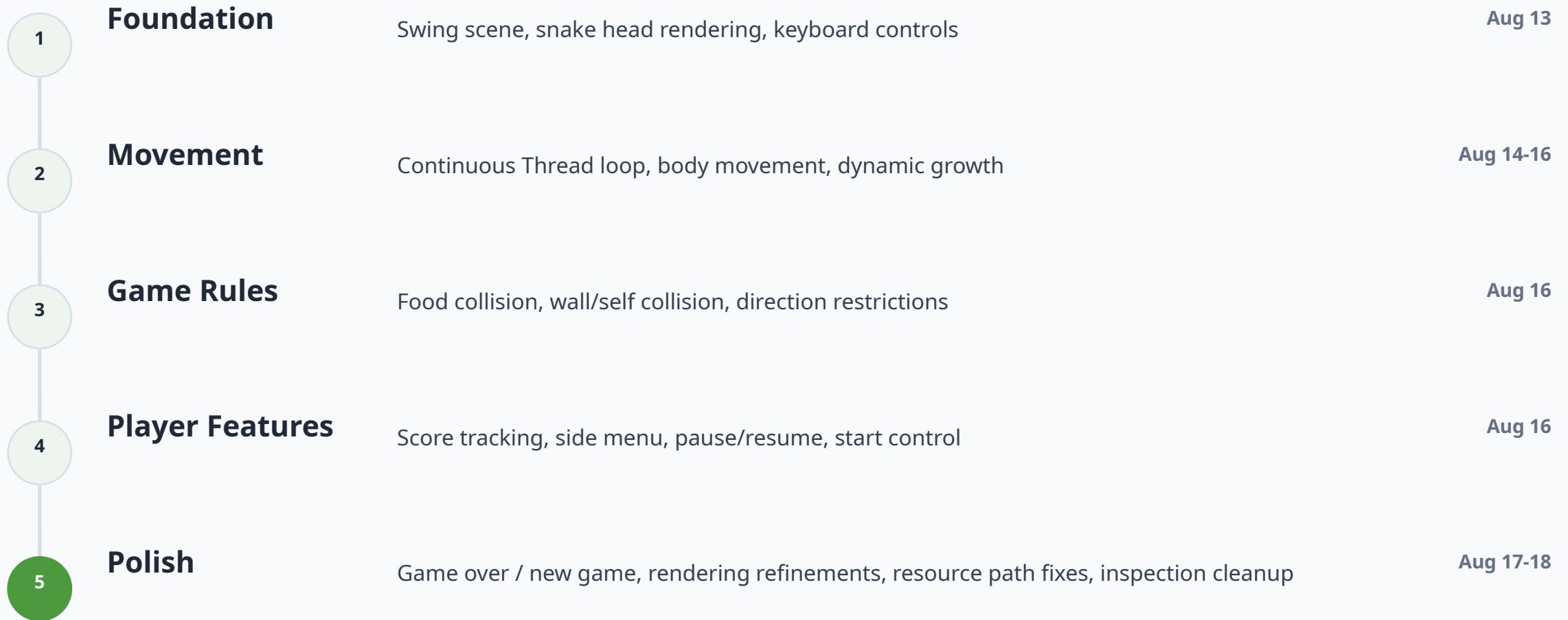
**3**

## Game Over / New Game

On a collision, a game-over overlay appears with the final score and a "New Game" control.

# Development Evolution

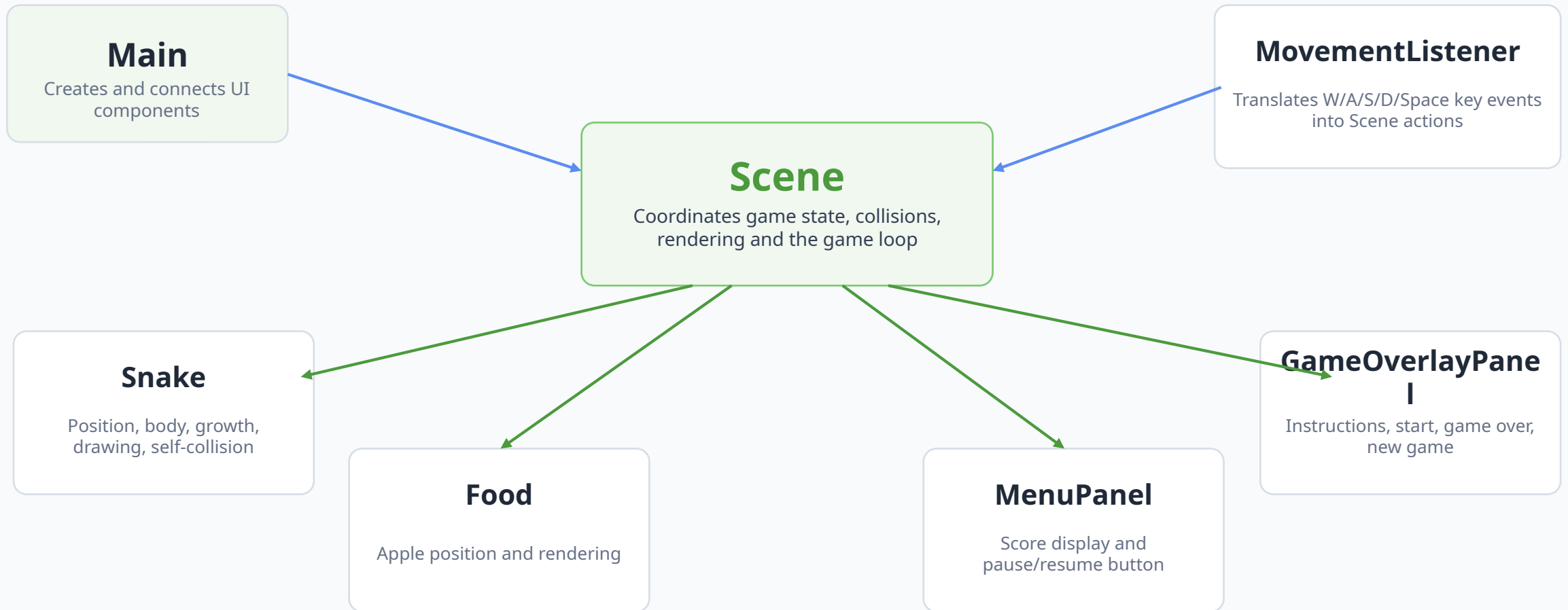
The Git history shows the game growing from a basic scene into a complete playable loop



*Latest commit included: "Resolve scene and snake inspection warnings" - a final code-quality cleanup rather than a new gameplay feature.*

# Object-Oriented Structure

Each class owns a clear responsibility instead of placing all game logic in one file



Composition / collaboration

# Object-Oriented Design in Practice

The structure improves encapsulation, readability and maintainability

## ENCAPSULATION

Internal fields are private. Other classes interact through methods such as `grow()`, `reset()`, `setPosition()`, `updateScore()` and `togglePause()`.

## SEPARATION OF CONCERNS

Rendering, input, game state and UI controls are split across dedicated classes instead of being mixed together.

## REUSABLE BEHAVIOR

Snake movement is expressed through clear methods: `moveRight()`, `moveLeft()`, `moveUp()` and `moveDown()`.

## EXAMPLE: THE SNAKE CLASS OWNS ITS OWN BODY STATE

```
private final ArrayList<Point> bodyParts;
private int desiredBodyLength;

public void grow() {
    desiredBodyLength++;
}

public boolean hasSelfCollision() {
    for (Point bodyPart : bodyParts) {
        if (x == bodyPart.x && y == bodyPart.y) {
            return true;
        }
    }
    return false;
}
```

## WHY THIS MATTERS

- Scene does not need to know how the snake stores or draws every body segment.
- Changing snake behavior is localized mainly inside `Snake.java`.
- UI classes can evolve without rewriting the movement logic.
- The design is easier to debug because each class has a focused role.

# Thread-Based Game Loop

A dedicated Thread keeps the snake moving continuously without using Swing Timer



## REAL GAME LOOP (SIMPLIFIED)

```
Thread gameThread = new Thread(() -> {
    while (true) {
        if (direction != null && !paused) {
            if (willHitWall()) { gameOver(); break; }

            moveSnake();

            if (snake.hasSelfCollision()) {
                gameOver();
                break;
            }

            if (snake.getX() == food.getX() &&
                snake.getY() == food.getY()) {
                snake.grow();
                score++;
                moveFoodToRandomPosition();
            }
        }
        repaint();
        Thread.sleep(300);
    }
});
gameThread.start();
```

## WHY A THREAD?

- The snake continues moving even when the player is not pressing a key.
- Keyboard input only changes direction; it does not perform each movement step.
- Pause/resume is implemented by checking a shared paused flag.
- The loop follows the project requirement to use Thread instead of Timer.
- Swing UI updates such as the game-over overlay are scheduled safely with `SwingUtilities.invokeLater()`.

# Collision Detection & Game Rules

Several checks work together to keep the game state valid



## Wall collision

Predict the next tile before movement. If the head would cross the border, the game ends.

```
willHitWall()
```



## Self collision

After movement, compare the head coordinates with every stored body point.

```
hasSelfCollision()
```



## Food collision

If the head and apple coordinates match, grow the snake, add score, and move the apple.

```
snake.getX() ==  
food.getX()
```



## Valid food spawn

Generate random grid positions until the new position is not occupied by the snake.

```
occupiesPosition(x, y)
```



## Direction protection

When the snake has a body, immediately reversing direction is ignored.

```
setDirection(...)
```

# Project Requirements - Implementation Map

The current version covers the required gameplay and two additional features

|                                         |                                                |      |
|-----------------------------------------|------------------------------------------------|------|
| <b>Swing game window</b>                | Main + Scene + layered panels                  | DONE |
| <b>Separate instructions screen</b>     | GameOverPanel with goal and controls           | DONE |
| <b>Start only after "Start Game"</b>    | Overlay button calls startNewGame()            | DONE |
| <b>Custom game characters / objects</b> | Snake head image + apple image + drawn body    | DONE |
| <b>Keyboard interaction</b>             | MovementListener for W/A/S/D + Space           | DONE |
| <b>Collision detection</b>              | Wall, self, food and valid spawn checks        | DONE |
| <b>Thread-based movement</b>            | Thread game loop + Thread.sleep(300)           | DONE |
| <b>Object-oriented design</b>           | Separate classes with focused responsibilities | DONE |

**Additional features implemented: SCORE TRACKING + PAUSE / RESUME**

# Project Summary

A small game that demonstrates UI, OOP, concurrency and game-state logic working together

- |    |                                |                                                                        |
|----|--------------------------------|------------------------------------------------------------------------|
| 01 | <b>Playable Swing Game</b>     | Custom snake and food rendering, score, start/pause/new-game flow      |
| 02 | <b>Clear OOP Structure</b>     | Focused classes make the project easier to understand and maintain     |
| 03 | <b>Thread-Based Movement</b>   | Automatic continuous movement implemented with Thread, not Timer       |
| 04 | <b>Robust Game Rules</b>       | Wall, self and food collision handling plus safe random food placement |
| 05 | <b>Incremental Development</b> | Git history documents each feature, fix and final cleanup step         |

**READY FOR LIVE DEMO**